

Manual técnico — Embed Experience

Documento-fonte conceitual para o time entender, operar e evoluir a plataforma Embed Experience. Descreve a arquitetura por responsabilidades — camadas, contratos, fluxos e padrões — sem amarrar conceitos a caminhos de arquivo. O objetivo é que qualquer engenheiro consiga raciocinar sobre o sistema independentemente de como o código está fisicamente organizado.

Estrutura: cada subsistema é apresentado com propósito → conceitos-chave → comportamento → como evoluir.

Parte 1 — Visão geral e contexto

1.1 O que é o Embed Experience

O Embed Experience é um template white-label multi-tenant que entrega uma experiência mobile embarcada via WebView. Cada parceiro embute um SDK nativo que abre uma EmbedView apontando para o BFF. O mesmo backend serve várias marcas, com tema e comportamento configurados por tenant.

A arquitetura tem três grandes pedaços: 1. **App nativo do parceiro**: hospeda o SDK e fornece o token de acesso. 2. **SDK nativo (EmbedView)**: abre o WebView, carrega a URL do BFF, injeta o token via bridge JavaScript. 3. **BFF (Backend-for-Frontend)**: renderiza a UI por tenant, decide entre dados reais (Core API) ou fictícios (mock), e expõe SDUI + bridge para o WebView.

1.2 Quem usa

Usuário	Como interage
Parceiro integrador	Embute a EmbedView no app deles, passa token + tenant. Não escreve código do BFF.
Cliente final (usuário do parceiro)	Usa o app do parceiro; ao abrir a experiência embarcada, vê a EmbedView carregada com o tema do parceiro.
Time da plataforma	Mantém o BFF, adiciona operações, customiza tenants, ajusta fixtures de mock.
QA do parceiro	Usa o modo sandbox (marker no lugar do JWT) para testar a UI sem credenciais reais.

1.3 Stack conceitual

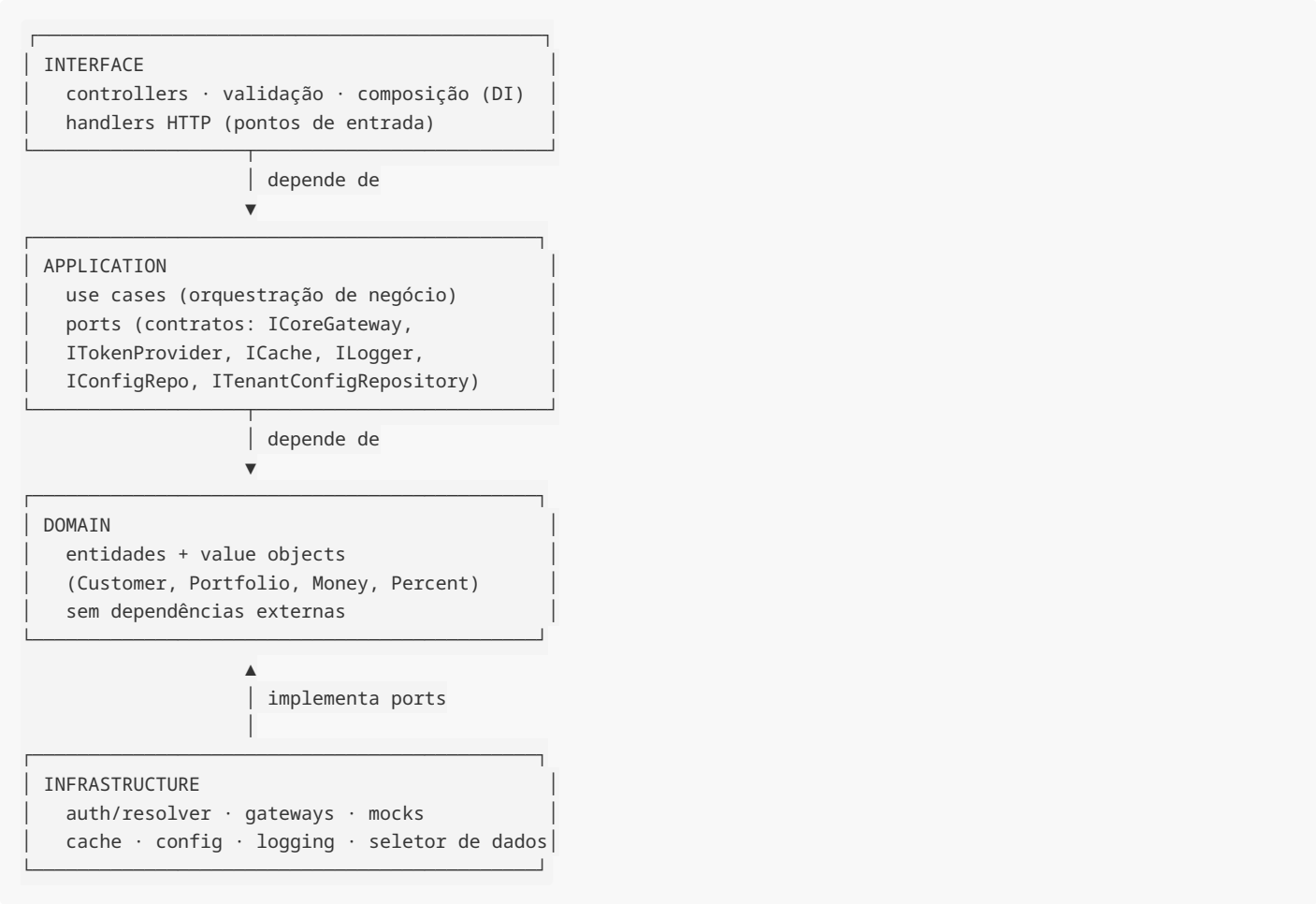
Domínio	Tecnologia	Papel
BFF	Next.js (App Router) + TypeScript estrito	Renderização por tenant, API HTTP, SDUI
Composição	Container de DI com escopo por request	Resolução de dependências isolada por chamada
Organização	Clean Architecture	Camadas domain → application → infra → interface
UI dinâmica	SDUI (templates JSON)	Telas dirigidas pelo servidor
Estilo	CSS vars por tenant	Tematização declarativa
Testes	Unit/integração + E2E	Garantia de contrato e fluxo
SDK nativo	WebView + bridge custom	Embarque + injeção de token

1.4 Princípios arquiteturais

- **Separação BFF / native:** o app nativo é só o casco; o BFF dirige a UI via SDUI. Mudanças visuais não exigem rebuild do app nativo.
- **Server-Driven UI:** a forma das telas mora em templates versionados. O cliente é um interpretador.
- **Modo decidido por request:** cada chamada ao BFF resolve sandbox vs live exclusivamente pelo formato do token. Sem flag global, sem fallback dinâmico.
- **Clean Architecture rigorosa:** a regra de dependência é respeitada — o domínio não importa nada; a aplicação depende só de ports; a infra implementa ports; a interface depende da aplicação.
- **Multi-tenancy por configuração:** cada tenant é um descritor declarativo. Tema, layout e fonte de dados são declarados, não codificados.
- **Composição auto-gerada:** o registro de DI é derivado das classes existentes por um gerador — evita edição manual e dessincronização.
- **Observabilidade primeira:** logs estruturados carregam env, tenantId, requestId, mockProfile. Dashboards filtram sandbox fora de produção.
- **Onboarding sem fricção:** o parceiro avalia o SDK sem credenciais via marker `app_sandbox_*`, alinhado ao padrão de chaves de teste de Stripe e Plaid.

Parte 2 — Arquitetura em camadas (Clean Architecture)

2.1 As quatro camadas



Regra de dependência: as setas só apontam para dentro (camadas mais estáveis). Quebrar isso é bug arquitetural.

2.2 Domain

Camada de regras de negócio puras — entidades e value objects sem dependências externas.

Conceito	Papel
Customer	Identificador + atributos básicos do usuário
Portfolio	Agregado de posições

Money	Valor monetário com formatação consistente
Percent	Percentual com precisão controlada

O domínio não conhece nenhuma outra camada nem bibliotecas externas além de tipos primitivos.

2.3 Application

A aplicação contém os contratos (ports) e a orquestração (use cases). Não conhece HTTP, JSON ou detalhes de transporte.

2.3.1 Ports (contratos)

Porta	Responsabilidade
ICoreGateway	Contrato com a API externa de negócio — ~17 operações (<code>getBalance</code> , <code>getCatalogPortfolios</code> , <code>postInvestAmount</code> , etc.)
ITokenProvider	Obter JWT para chamadas service-to-service, com cache e renovação proativa
ICache	Get/set/delete com TTL e operações em lote
IConfigRepo	Acesso a configuração por chave, incluindo settings por tenant
ILogger	Logging estruturado (debug/info/warn/error) com contexto
ITenantConfigRepository	Resolver o descritor de um tenant a partir do slug

2.3.2 Use cases

Cada use case é uma classe injetável que orquestra uma operação de negócio. O conjunto cobre a superfície da API HTTP:

Use case	Operação HTTP
GetBalanceUseCase	GET /api/balance
GetInvestFlowUseCase	GET /api/invest/flow
GetInvestReviewUseCase	GET /api/invest/review
PostInvestAmountUseCase	POST /api/invest/amount
PostInvestIntentionUseCase	POST /api/invest/intention
GetInvestmentsCategoryUseCase	GET /api/investments/category
GetInvestmentsProductsUseCase	GET /api/investments/products
GetInvestmentsProductsSummaryUseCase	GET /api/investments/products-summary
GetCatalogCategoryUseCase	GET /api/catalog/category
GetCatalogProductDetailsUseCase	GET /api/catalog/product-details
GetCatalogPortfoliosUseCase	GET /api/catalog/portfolios
GetPortfolioBuilderRiskSelectUseCase	GET /api/portfolio-builder/risk-select
GetPortfolioBuilderPreviewUseCase	GET /api/portfolio-builder/preview
PostPortfolioBuilderCreatePortfolioUseCase	POST /api/portfolio-builder/create
GetPortfolioHeroUseCase	GET /api/portfolio-details/hero

GetConsolidatedCustodyUseCase	GET /api/consolidated-custody
GetUserWalletsUseCase	GET /api/wallets

2.3.3 Padrão conceitual de Use Case

```
@injectable()
export class GetBalanceUseCase {
  constructor(
    @inject(ICORE_GATEWAY_TOKEN) private gateway: ICoreGateway,
  ) {}

  async execute(input: GetBalanceInput): Promise<GetBalanceOutput> {
    // Validação fica no controller. Aqui é orquestração.
    return this.gateway.getBalance(input);
  }
}
```

Use cases são finos por design — a maioria é um proxy ao gateway. Lógica de domínio (cálculos, regras) entra aqui quando existir.

2.4 Infrastructure

Implementações concretas dos ports e integrações externas.

2.4.1 Gateways

Gateway	Quando é usado
CoreHttpGateway	Modo live — chama a Core API real, injetando o <code>accessToken</code> .
CoreMockGateway	Modo sandbox ou tenant com fonte de dados mock — devolve fixtures conforme o <code>MockProfile</code> .

2.4.2 Mocks por operação

Cada operação do gateway tem uma ou mais funções de fixture (caminho feliz e, opcionalmente, estado vazio), reconciliadas pelo helper universal `withProfile`:

```
export async function withProfile<T>(
  profile: MockProfile | undefined,
  fns: { happyPath: () => T; empty?: () => T; operation?: string },
): Promise<T> {
  switch (profile) {
    case "error": throw new MockGatewayError(fns.operation);
    case "slow":  await delay(3000); return await fns.happyPath();
    case "empty": return fns.empty ? await fns.empty() : await fns.happyPath();
    default:     return await fns.happyPath();
  }
}
```

`error` e `slow` são universais (funcionam para qualquer operação). `empty` exige uma fixture explícita por operação; sem ela, cai em `happyPath` com warning.

2.4.3 Auth / Resolver

A camada de autenticação resolve o modo de execução a partir do token. Conceitos:

Conceito	Responsabilidade
ExecutionContext	Objeto com mode, tenantId, customerId, mockProfile? — propagado por DI

parseSandboxMarker	Parser estrito do marker de sandbox (+ fast-path <code>isSandboxMarker</code>)
decodeJwtPayload	Decoder base64url do payload do JWT, sem verificação de assinatura
assertTenantExists	Valida o slug contra os tenants registrados
resolveExecutionMode	Orquestrador que ramifica entre marker (sandbox) e JWT (live)

2.4.4 Seletor de fonte de dados

Mantém o estado de configuração de tenants em memória e expõe a resolução do descritor:

```
export function getTenantConfig(tenantId: string): TenantConfig {
  return tenantConfigStore.getConfig(tenantId);
  // → { dataSource: "mock" | "core" }
}
```

2.5 Interface

2.5.1 Controllers

Padrão uniforme em todos eles: validar → logar → executar → responder.

```
@injectable()
export class BalanceController {
  constructor(
    @inject(GET_BALANCE_USE_CASE_TOKEN) private getBalanceUseCase: GetBalanceUseCase,
    @inject("executionContext") private executionContext: ExecutionContext,
  ) {}

  async getBalance(request: NextRequest): Promise<NextResponse> {
    const context = createRequestContext(request, { env: this.executionContext.mode });
    logger.logRequestStart(context);

    try {
      const validation = GetBalanceSchema.safeParse({ customerId: /* ... */ });
      if (!validation.success) {
        return createSecureErrorResponse(request, "Validation failed");
      }

      const result = await this.getBalanceUseCase.execute(validation.data);
      logger.logBusinessEvent(context, "balance_retrieved", { netWorth: result.netWorth });

      return NextResponse.json(result, { status: 200 });
    } catch (error) {
      logger.logRequestError(context, error as Error);
      return createSecureErrorResponse(request, "Internal server error");
    }
  }
}
```

2.5.2 Validação

Schemas declarativos rodam no controller, antes do use case. Falhas viram HTTP 422.

```
export const GetBalanceSchema = z.object({
  customerId: z.string().min(1, "Customer ID is required"),
});
```

2.5.3 Composição (DI)

Os tokens de DI e o registro do container são auto-gerados a partir das classes existentes. A regra conceitual é: não editar o registro à mão — alterar o gerador e regenerar após adicionar use case ou controller.

2.6 Dependency Injection

2.6.1 Root container vs request container

- **Root container:** inicializado uma vez no boot do BFF. Registra controllers, use cases e singletons de infra.
- **Request container:** um child container criado a cada request, com escopo da chamada. Recebe o `ExecutionContext` resolvido.

```
export function createRequestContainer(
  ctx: ExecutionContext,
  accessToken: string = "",
): DependencyContainer {
  const requestContainer = container.createChildContainer();
  registerServicesInContainer(requestContainer, ctx);

  requestContainer.registerInstance("executionContext", ctx);
  requestContainer.registerInstance("tenantId", ctx.tenantId);
  requestContainer.registerInstance("customerId", ctx.customerId);
  requestContainer.registerInstance("accessToken", accessToken);

  return requestContainer;
}
```

2.6.2 Precedência de seleção de gateway

Token recebido	Fonte de dados do tenant	Gateway resolvido
app_sandbox_*	(qualquer)	CoreMockGateway
JWT	mock	CoreMockGateway (tenant mock-only por config)
JWT	core	CoreHttpGateway (live)
qualquer (modo de teste)	(qualquer)	CoreMockGateway (override para testes)

2.6.3 Como adicionar um use case (conceitual)

1. Criar a classe do use case (injetável) na aplicação.
2. Criar o controller correspondente na interface.
3. Regenerar o registro de DI — o gerador detecta as classes e atualiza tokens + container.
4. Conectar o handler HTTP seguindo o padrão canônico de rota (§6.5.2).

Parte 3 — Multi-tenancy

3.1 Descritor de tenant

Cada tenant é um descritor declarativo. Exemplo conceitual:

```
{
  "id": "partnerco",
  "name": "Partner Co",
  "slug": "partnerco",
  "dataSource": "mock",
  "theme": {
    "primaryColor": "#1A56DB",
    "secondaryColor": "#9333EA",
    "accentColor": "#FFFFFF",
    "backgroundColor": "#F5F5F5",
    "logoUrl": "/imgs/partnerlogo.png"
  },
  "layout": {
```

```

"header": { "showLogo": false, "showTitle": false, "title": "Partner Co" },
"appBar": { "default": { }, "routes": { "/partnerco": { "backBehavior": "closeWebView" } } },
"navigation": { "showAppBar": true }
},
"security": { "corsOrigins": [], "ipAllowlist": [], "requireAuth": true },
"features": { "portfolios": true, "intents": true, "products": true },
"version": "0.0.1"
}

```

Campos-chave: * `dataSource` — "mock" ou "core". Define o gateway default (sobreposto pelo marker sandbox). * `theme` — cores e logo, aplicados via CSS vars. * `layout` — comportamento do app bar (sticky, frosted, back behavior por rota).

3.2 Roteamento dinâmico por tenant

Cada tenant tem um segmento de rota próprio (ex.: `/partnerco/invest`). O segmento de agrupamento de tenants é organizacional e não aparece na URL. A estrutura de páginas por tenant espelha a árvore de telas (home, invest, portfolios, portfolio-builder, etc.).

3.3 Tematização

Um provider de tema client lê o `theme` do descritor e aplica CSS vars no `:root`:

```

useLayoutEffect(() => {
  const root = document.documentElement;
  root.style.setProperty("--tenant-primary", "26 86 219"); // RGB
  root.style.setProperty("--tenant-primary-rgb", "26, 86, 219");
  root.style.setProperty("--tenant-secondary", "147 51 234");
  root.style.setProperty("--tenant-canvas", "#F5F5F5");
  root.style.setProperty("--tenant-logo-url", "url(/imgs/partnerlogo.png)");
}, [theme]);

```

O CSS consome via variáveis:

```
<div className="bg-[rgb(var(--tenant-primary-rgb))] text-white">...</div>
```

3.4 Layout compartilhado de tenant

Um layout de segmento de tenant é o gancho único que aplica, em todas as rotas daquele tenant: o provider de tema, o banner de sandbox, o app bar global e o corpo rolável.

```

export async function TenantSegmentLayout({ slug, children }) {
  const tenantConfig = getTenantConfig(slug);
  return (
    <TenantThemeProvider initialTheme={tenantConfig.theme}>
      <SandboxBanner />
      <GlobalAppBar config={tenantConfig.layout.appBar} />
      <TenantAppBarScrollBody>{children}</TenantAppBarScrollBody>
    </TenantThemeProvider>
  );
}

```

3.5 Precedência mock vs core

Token recebido	Fonte de dados	Modo efetivo
<code>app_sandbox_*</code>	(qualquer)	sandbox
JWT válido	mock	sandbox*
JWT válido	core	live

* O tenant é "mock-only" por configuração.

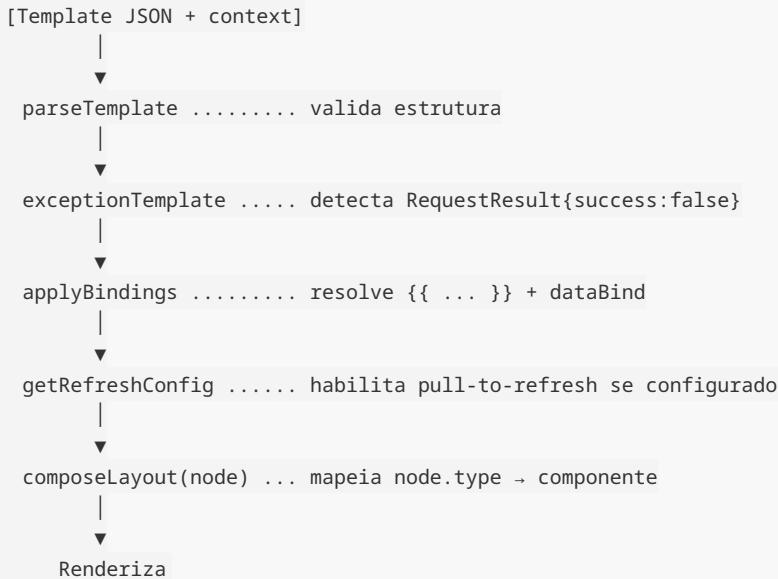
Parte 4 — Server-Driven UI (SDUI)

4.1 Conceito

SDUI é um DSL JSON que descreve a UI. O servidor envia o template, o client interpreta e renderiza. Mudanças visuais não exigem rebuild do app nativo — basta atualizar o template no BFF.

Benefícios: * Tenants visualmente diferentes sem código branched. * A/B testing trocando templates. * Hotfix de UI sem release de app.

4.2 Pipeline de renderização



4.3 Estrutura de template

```
{
  "templateId": "invest_categoria_v1",
  "version": "2026-04-22",
  "layout": {
    "sections": [
      {
        "type": "container",
        "props": { "fullHeight": true },
        "body": [
          { "type": "tabs", "props": { "show": "{{ showTabs }}" }, "items": "{{ tabs }}" } }
        ]
      }
    ]
  }
}
```

O nó base:

```
type TemplateNode = {
  type: string;                // tipo de componente
  id?: string;                 // identificador único
  props?: Record<string, unknown>; // suportam {{ ... }}
  body?: TemplateNode[];      // filhos (containers)
  children?: TemplateNode[];   // filhos (lists com dataBind)
};
```

4.4 Data binding

Recurso	Exemplo	Resolve
Expressão simples	<code>{{ usuario.nome }}</code>	<code>context.usuario.nome</code> (dot notation)
Condicional	<code>{{ validacao.success && usuario.premium }}</code>	booleano para <code>show</code>
Pipe / filtro	<code>{{ saldo \ currency('BRL') }}</code>	aplica formatação
Loop (dataBind)	<code>{{ portfolio.positions }}</code>	gera N nós; dentro, <code>\$item</code> e <code>\$index</code>

Exemplo de loop:

```
{
  "type": "list",
  "props": { "dataBind": "{{ portfolio.positions }}" },
  "children": [
    { "type": "text", "props": { "value": "{{ $item.name }}" } },
    { "type": "text", "props": { "value": "R$ {{ $item.value | currency }}" } }
  ]
}
```

4.5 Registro de componentes

Um composer mapeia `node.type` → componente. ~40 tipos, agrupados por família:

- **Core UI:** `text`, `button`, `icon`, `input`, `row`, `container`
- **Cards:** `card-balance`, `category-card`, `product-card`, `portfolio-card`, `catalog-card`, `information-card`, `invest-card`, `product-performance-card`
- **Headers:** `main-header`, `product-header`, `product-details-header`, `catalog-header`
- **Listas:** `list`, `product-list`, `benefit-list`, `performance-list`
- **Especializados:** `window`, `tabs`, `pull-to-refresh`, `empty-state`, `exception-error`, `my-wallets-content`, `invest-amount`, `invest-review`, `portfolio-builder-catalog`
- **Por tenant:** componentes específicos de marca (ex.: `partnerco-balance-card`, `partnerco-quick-actions`)

4.6 Dispatcher + FlowEngine

4.6.1 Dispatcher

Sistema middleware-based que processa eventos de UI:

```
Dispatcher.dispatch(action)
  → middlewares[0..N]           // transformações em cadeia
  → hooks.beforeDispatch
    → handleEvent(action)
      └ type: "navigate" → FlowEngine.handle()
      └ type: "bridge"  → sendToBridge()
  → hooks.afterDispatch
```

4.6.2 Evento de UI (ButtonEvent)

```
{
  "label": "string",
  "type": "navigate | bridge",
  "target": "string",
  "event": "string",
  "payload": "unknown"
}
```

4.6.3 FlowEngine

Camada de handlers que reage aos eventos do dispatcher:

```
flowHandlers = {
  navigation: (target) => dispatchEvent("sdui:navigate"),
  navigateFlow: (target) => navega com prefixo,
  replaceCurrent: (target) => substitui o último segmento,
  back: () => dispatchEvent("sdui:back"),
  refreshHomePage: () => dispara refresh da home,
};
```

CustomEvents são ouvidos por wrappers de roteamento que executam *push/replace/back*.

4.7 Pull-to-refresh

Configuração por `templateId`. Quando existe, o renderer envolve o conteúdo num provider de refresh, habilitando o gesto nativo:

```
type RefreshConfig = {
  fetchData: () => Promise<string>;
  getInitialValue: (context) => string;
};
```

4.8 Exception templates

Quando a árvore de contexto contém um `RequestResult { success: false }`, o pipeline troca para um template alternativo de erro (componente `exception-error`):

```
{ "portfolio": { "success": false, "message": "API indisponível" } }
// → renderiza tela de erro com a mensagem
```

4.9 Como evoluir a SDUI (conceitual)

Novo componente: 1. Criar o componente de UI. 2. Criar o dispatcher associado, se reagir a eventos. 3. Registrar o tipo no composer. 4. Referenciar `"type": "<novo-tipo>"` nos templates.

Novo template: 1. Definir o JSON com `templateId`, `version`, `layout`. 2. Validar contra o schema de template. 3. Configurar refresh, se houver gesto pull-to-refresh. 4. Renderizar via `TemplateRenderer` com o contexto de dados.

Parte 5 — Bridge nativo ↔ Web

5.1 Conceito

A bridge é a camada de mensageria entre o WebView e o app nativo, inspirada em Ionic/Capacitor: * **Bidirecional:** web → nativo (requests) e nativo → web (events). * **Whitelist de métodos:** só métodos declarados no schema passam. * **Validação por payload:** schema no lado web; parsing no lado nativo. * **Versionada:** campo `bridgeVersion` em todo envelope.

5.2 Envelope protocol

Toda mensagem é um envelope JSON:

```
{
  "id": "UUID",
  "origin": "app | sdk",
  "type": "request | response | event | error",
  "method": "string",
  "params": {},
}
```

```

"result": {},
"error": { "code": "string", "message": "string" },
"meta": { "bridgeVersion": "1.0.0", "tenantId": "string" }
}

```

Correlação por UUID: o web posta um request com `id = uuid()` e guarda uma Promise pendente; o nativo responde com o mesmo id; o web resolve a Promise correspondente.

5.3 Métodos

5.3.1 Web → Nativo (request/response)

Método	Params	Result
<code>closeWebView()</code>	<code>{}</code>	<code>{ ok: true }</code>
<code>native.getEnvInfo()</code> <i>(planejado)</i>	<code>{}</code>	<code>{ platform, appVersion, osVersion }</code>
<code>native.getCustomerContext()</code> <i>(planejado)</i>	<code>{}</code>	<code>{ tenantId, customerId, mockProfile }</code>
<code>native.openNativeShare()</code> <i>(planejado)</i>	<code>{ text, url? }</code>	<code>{ ok: true }</code>
<code>native.haptics()</code> <i>(planejado)</i>	<code>{ type: "light" "medium" "heavy" }</code>	<code>{ ok: true }</code>

5.3.2 Nativo → Web (events)

Evento	Params
<code>tokenReady</code>	<code>{ hasToken: boolean }</code>
<code>app.themeChanged</code> <i>(planejado)</i>	<code>{ theme: "light" "dark" }</code>
<code>app.customerChanged</code> <i>(planejado)</i>	<code>{ customerId: string }</code>
<code>app.deeplink</code> <i>(planejado)</i>	<code>{ path: string, params?: Record<string,string> }</code>

5.4 Injeção de token

5.4.1 Produção (via SDK nativo)

Após a página carregar, o SDK injeta o token no escopo global e dispara o evento `tokenReady`:

```

(function () {
  window.__APP_ACCESS_TOKEN__ = "<token>";
  window.dispatchEvent(new CustomEvent("tokenReady", {
    detail: { hasToken: typeof window.__APP_ACCESS_TOKEN__ === "string" },
  }));
})();

```

5.4.2 Desenvolvimento (sem SDK)

Um loader dev-only permite injetar um token manualmente: 1. Lê um parâmetro de query (ex.: `?appToken=`) e injeta no escopo global. 2. Persiste o valor para os próximos reloads. 3. Remove o parâmetro da URL para não vazar no histórico.

Em produção, esse loader é inerte — só o SDK injeta.

5.5 Validação de protocolo

Schema do envelope e do profile de mock, compartilhado entre detecção de banner (client) e o contrato de bridge:

```
export const EnvelopeSchema = z.object({
  id: z.string().uuid(),
  origin: z.enum(["app", "sdk"]),
  type: z.enum(["request", "response", "event", "error"]),
  method: z.string().optional(),
  params: z.record(z.string(), z.any()).optional(),
  result: z.any().optional(),
  error: z.object({ code: z.string(), message: z.string() }).optional(),
  meta: z.object({ bridgeVersion: z.string(), tenantId: z.string() }),
});

export const MockProfileSchema = z.enum(["happyPath", "empty", "error", "slow"]);
```

5.6 Fluxo completo de uma mensagem

```
[Web] EmbedBridge.request("closeWebView", {})
      | uuid = v4(); pendingRequests[uuid] = Promise
      ▼
      window.postMessage(envelope)
      |
      ▼
[Nativo] canal recebe → MessageHandler.handle(message)
        | switch(method) → "closeWebView"
        ▼
        ação nativa (fechar a view)
        |
        ▼
        devolve { id: uuid, type: "response", result: { ok: true } }
        |
        ▼
[Web] listener captura → handleResponse(envelope)
      | pendingRequests[uuid].resolve(result)
      ▼
      Promise no caller resolve
```

Parte 6 — Sistema de Sandbox (decisão arquitetural)

6.1 Por que existe

Parceiros precisam validar a integração do SDK — onboarding, demo, debug, regressão — sem depender do ambiente real e na mesma URL de produção.

Sem o sandbox, o parceiro precisaria implementar service-to-service, obter credenciais, autenticar e só então avaliar a UI. Com o sandbox: basta passar um marker no lugar do JWT. Sem credenciais — alinhado às chaves de teste de Stripe e Plaid.

6.2 Formato do marker

`app_sandbox_<tenant>_<profile>`

Regex estrita: `^app_sandbox_(?<tenant>[a-z0-9-]{1,32})_(?<profile>happyPath|empty|error|slow)$`

Exemplos: `app_sandbox_partnerco_happyPath`, `app_sandbox_partnerco_empty`, `app_sandbox_sample_error`, `app_sandbox_example-co_slow`. *Tokens malformados → HTTP 400.*

6.3 Os 4 profiles

Profile	Comportamento
happyPath	Fixtures ricas — usuário com posição. Default.
empty	Listas vazias, valores zerados — conta sem histórico.

error	Toda operação lança <code>MockGatewayError</code> → HTTP 500.
slow	Delay de 3s antes de retornar <code>happyPath</code> .

6.4 Fase 1 — Resolver

O resolver decide o modo de execução a partir do token, uma vez por request:

```
export function resolveExecutionMode(token: string): ExecutionContext {
  if (isSandboxMarker(token)) {
    const parsed = parseSandboxMarker(token);
    if (!parsed) throw new Error("invalid_sandbox_marker");
    assertTenantExists(parsed.tenant);
    return {
      mode: "sandbox",
      tenantId: parsed.tenant,
      customerId: `cus_test_${parsed.profile}_${parsed.tenant}`,
      mockProfile: parsed.profile,
    };
  }
  const claims = decodeJwtPayload(token);
  if (!claims.sub) throw new Error("missing_subject");
  return {
    mode: "live",
    tenantId: typeof claims.tenant === "string" ? claims.tenant : "",
    customerId: claims.sub,
  };
}
```

Erros mapeados: invalid_sandbox_marker → 400 · unknown_tenant → 400 · missing_subject → 401 · invalid_jwt → 401.

6.5 Fase 2 — DI por modo

6.5.1 Helper de resolução de contexto

Combina três operações em cada rota: extrair o bearer token, resolver o tenant do header, e resolver o modo de execução — com um cross-check exclusivo do sandbox.

```
export function resolveRequestContext(request: NextRequest):
  ResolvedRequestContext | Response {
  const accessToken = extractBearerToken(request);
  const requestTenantId = resolveTenant(request);

  let ctx: ExecutionContext;
  try {
    ctx = resolveExecutionMode(accessToken);
  } catch (err) { return jsonError(400 /* | 401 */); }

  // Cross-check: exclusivo do sandbox
  if (ctx.mode === "sandbox" && ctx.tenantId !== requestTenantId) {
    return jsonError(403, "tenant_mismatch");
  }
  return { ctx, accessToken, requestTenantId };
}
```

6.5.2 Padrão canônico de handler HTTP

Todo handler de rota segue este shape:

```
export async function GET(request: NextRequest) {
  try {
    const resolved = resolveRequestContext(request);
    if (resolved instanceof Response) return resolved; // 400/401/403
    const { ctx, accessToken, requestTenantId: tenantId } = resolved;
```

```

const container = createRequestContainer(ctx, accessToken);
const controller = container.resolve<XController>(X_CONTROLLER_TOKEN);
return await controller.handler(request, tenantId);
} catch (error) { /* trata 500 */ }
}

```

6.5.3 Cross-check de tenant

Regra exclusiva do sandbox: o tenantId do marker precisa bater com o tenant declarado no header (`x-tenant-id`); mismatch → 403. Em live, o slug é a fonte da verdade e não há cross-check.

6.5.4 Observabilidade

O contexto de request carrega o campo `env` (o modo de execução). Os logs estruturados o incluem, permitindo que dashboards de produção filtrem `env != 'sandbox'`.

6.6 Fase 3 — Fixtures por profile

O gateway de mock injeta o `ExecutionContext` e roteia cada operação pelo `withProfile`:

```

@injectable()
export class CoreMockGateway implements ICoreGateway {
  constructor(
    @inject("executionContext") private executionContext: ExecutionContext,
  ) {}

  private get profile() { return this.executionContext.mockProfile; }

  async getBalance(): Promise<GetBalanceOutput> {
    return withProfile(this.profile, {
      operation: "getBalance",
      happyPath: () => getBalanceMock(),
      empty: () => getBalanceMockEmpty(),
    });
  }
  // ... demais operações
}

```

Cobertura de estado vazio: hoje 4 das ~18 operações têm fixture empty explícita (saldo, carteiras, portfólios de catálogo, custódia consolidada). As demais caem em happyPath com warning até receberem a sua própria fixture.

6.7 Fase 4 — Banner de sandbox

Um hook (`useSandboxMode`) detecta o marker no escopo global e reage ao evento `tokenReady`:

```

const SANDBOX_MARKER_RE =
  /^app_sandbox_([a-z0-9-]{1,32})_(happyPath|empty|error|slow)$/;

export function useSandboxMode(): SandboxMode {
  const [mode, setMode] = useState<SandboxMode>(() => inspect(readToken()));
  useEffect(() => {
    const update = () => setMode(inspect(readToken()));
    update();
    window.addEventListener("tokenReady", update);
    return () => window.removeEventListener("tokenReady", update);
  }, []);
  return mode;
}

```

O SandboxBanner é um banner âmbar persistente e não-fechável por design — a mitigação primária do risco de "promover um marker para produção sem perceber". Ele é montado no layout de segmento de tenant, cascadeando para todas as páginas.

6.8 Fase 6 — Suíte E2E

Um helper de login injeta o marker antes do JS da página rodar, o que funciona independentemente do ambiente:

```
export async function loginAsSandbox(page, { tenant, profile }): Promise<string> {
  const token = `app_sandbox_${tenant}_${profile}`;
  await page.addInitScript((t) => { window.__APP_ACCESS_TOKEN__ = t; }, token);
  return token;
}
```

A suíte cobre visibilidade do banner em todos os tenants × 4 profiles, acessibilidade, estado live e marker malformado. Atualmente suspensa enquanto o ambiente de CI não suporta os pré-requisitos visuais.

6.9 Estado e roadmap das fases

```
Sandbox via token marker
├─ F1 Resolver + parser
├─ F2 DI por modo
├─ F3 Fixtures por profile
├─ F4 Banner + DX
├─ F5 🚧 Hardening (verificação de assinatura, rate limit)
└─ F6 🚧 E2E (suíte suspensa)
```

Próximos passos: fixtures empty para as operações restantes; hardening (F5) sob demanda; reativação da suíte E2E.

Parte 7 — Fluxo end-to-end

7.1 Fluxo live

1. Backend do parceiro → service-to-service com o provedor → JWT
2. App parceiro abre `EmbedView(token: jwt, tenant: "partnerco")`
3. `EmbedView` abre o `WebView` na URL do tenant
4. Bridge injeta: `window.__APP_ACCESS_TOKEN__ = jwt; dispatchEvent("tokenReady")`
5. Banner: ausente (live)
6. Chamada `GET /api/balance:`
7. `Authorization: Bearer <jwt>`
8. `x-tenant-id: partnerco`
9. Handler HTTP → `resolveRequestContext:`
10. JWT → `decodeJwtPayload` → `{ sub, tenant? }`
11. `ctx = { mode: "live", tenantId, customerId: sub }`
12. cross-check: pulado (não é sandbox)
13. `createRequestContainer` → registra `CoreHttpGateway`
14. Controller valida e chama o use case
15. `CoreHttpGateway` encaminha para a Core API
16. Resposta → mapper → JSON para o client

7.2 Fluxo sandbox

1. Parceiro decide testar sem credenciais
2. App parceiro abre `EmbedView(token: "app_sandbox_partnerco_happyPath", tenant: "partnerco")`
3. `EmbedView` abre o `WebView` (idem live)
4. Bridge injeta: `window.__APP_ACCESS_TOKEN__ = "app_sandbox_partnerco_happyPath"`
5. `SandboxBanner` detecta o marker → banner âmbar aparece
6. Chamada `GET /api/balance:`
7. `Authorization: Bearer app_sandbox_partnerco_happyPath`
8. `x-tenant-id: partnerco`
9. Handler HTTP → `resolveRequestContext:`

- 10. `isSandboxMarker` → `true`
- 11. `parse` → `{ tenant: "partnerco", profile: "happyPath" }`
- 12. `assertTenantExists` → `ok`
- 13. `ctx` = `{ mode: "sandbox", tenantId: "partnerco", customerId: "cus_test_happyPath_partnerco", mockProfile: "happyPath" }`
- 14. cross-check: marker tenant === header tenant
- 15. `createRequestContainer` → registra `CoreMockGateway`
- 16. Controller chama o use case
- 17. `CoreMockGateway` → `withProfile("happyPath", { happyPath: getBalanceMock })`
- 18. Resposta → mapper → JSON para o client
- 19. Logs: `env="sandbox", mockProfile="happyPath", tenantId="partnerco"`

7.3 Exemplo: home com sandbox empty

- 1. Token sandbox com profile `"empty"` injetado no tenant
- 2. Banner âmbar aparece: "Modo sandbox ... empty"
- 3. A home dispara a busca de custódia consolidada
- 4. Gateway resolve sandbox + empty: `typescript withProfile("empty", { happyPath: () => getConsolidatedCustodyMock(true), empty: () => getConsolidatedCustodyMockEmpty(true), })` *Empty fixture:* `{ isEmpty: true, totalValue: 0, managedPortfolios: [] }`
- 5. UI mostra:
- 6. Banner âmbar
- 7. Card de custódia vazio ("Comece a investir")
- 8. Quick actions: fallback para happyPath (fixture empty ainda não existe → warning)

Inconsistência de UX a corrigir: custódia vazia + quick actions populadas. Resolução: adicionar a fixture empty que falta para as quick actions.

Parte 8 — Ciclo de desenvolvimento

8.1 Injeção de marker para teste manual

Há três formas conceituais de colocar o sistema em sandbox durante o desenvolvimento:

Forma	Como	Escopo
Query param	Acrescentar <code>?appToken=app_sandbox_<tenant>_<profile></code> à URL	Dev-only; o loader injeta e limpa a URL
Persistência local	Gravar o marker no armazenamento local do browser	Sobrevive a reloads
Escopo global direto	Atribuir o token a <code>window.__APP_ACCESS_TOKEN__</code> e disparar <code>tokenReady</code>	Só a sessão atual

8.2 Modelo de branches e PRs

```
main
├─ staging
│   └─ epic/<slug>
│       └─ feat/<epic>/<feature>
│           ├── commit (refs sub-issue)
│           └─ PR: feature → epic
```

O PR é sempre da branch de feature, não por sub-issue. Sub-issues viram commits referenciados.

Tipo	Padrão
Epic	<code>epic/<slug></code>

Feature	feat/<epic-slug>/<feature-slug>
Fix	fix/<area>/<slug>
Chore	chore/<slug>
Hotfix	hotfix/<slug>

Commits: *tipo(escopo): descrição, com tipos feat, fix, chore, docs, refactor, test, build, ci.*

8.3 Definition of Done por PR

- Todas as sub-issues commitadas e referenciadas.
- Lint, typecheck e testes verdes.
- Documentação afetada atualizada no mesmo PR.
- PR linkado à feature e ao epic.
- Sem segredos no diff.
- Sem dependências novas não-aprovadas.
- Sem mudança na camada de segurança sem decisão arquitetural registrada.

8.4 Guardrails de segurança

Nunca: * Push direto em main ou staging; force-push em branches compartilhadas. * Apagar branches remotas sem confirmação explícita. * Commitar segredos. * Atravessar a fronteira Mock ↔ Core na mesma execução. * Cair em sandbox por default em produção. * Instalar dependências novas sem aprovação. * Alterar a camada de segurança sem decisão arquitetural registrada.

Sempre: * Uma branch isolada por tarefa; PR pequeno e focado. * Marcar explicitamente qualquer trecho sandbox-only. * CI verde antes de marcar o PR como pronto. * Atualizar a documentação no mesmo PR. * Pré-flight para ações irreversíveis: listar o que será afetado → repetir a intenção em uma frase → aguardar confirmação humana explícita → só então executar.

Parte 9 — Como evoluir o sistema

9.1 Adicionar uma operação nova (percurso pelas camadas)

Para criar uma operação GET /api/foo, você percorre as camadas de fora para dentro do contrato e de volta:

#	Camada	Ação conceitual
1	Application (port)	Declarar <code>getFoo(input): Promise<GetFooOutput></code> no <code>ICoreGateway</code>
2	Application (use case)	Criar <code>GetFooUseCase</code> que orquestra a operação
3	Infra (mocks)	Criar fixtures <code>getFooMock</code> e <code>getFooMockEmpty</code>
4	Infra (gateway live)	Implementar <code>getFoo</code> no <code>CoreHttpGateway</code>
5	Infra (gateway mock)	Implementar <code>getFoo</code> no <code>CoreMockGateway</code> via <code>withProfile</code>
6	Interface (validação)	Definir o schema de entrada
7	Interface (controller)	Criar <code>FooController</code> no padrão validar → logar → executar → responder
8	Interface (rota)	Adicionar o handler HTTP no padrão canônico
9	Composição	Regenerar o registro de DI
10	Testes	Cobrir use case, controller e os 4 profiles do mock
11	Docs	Atualizar a referência de comportamento por profile

Padrão do gateway mock no passo 5:

```
async getFoo(input: GetFooInput): Promise<GetFooOutput> {
  return withProfile(this.profile, {
    operation: "getFoo",
    happyPath: () => getFooMock(),
    empty: () => getFooMockEmpty(),
  });
}
```

9.2 Adicionar um componente SDUI

- 1. Criar o componente de UI e, se reagir a eventos, o dispatcher.
- 2. Registrar o tipo no composer.
- 3. Referenciar o tipo nos templates.
- 4. Testar o dispatcher quando houver lógica.

9.3 Adicionar um tenant

- 1. Criar o descritor do tenant a partir de um existente, validando contra o schema.
- 2. Criar as rotas do segmento daquele tenant, envolvendo-as no layout de segmento.
- 3. A validação de tenant passa a aceitar o novo automaticamente, pois lê dos tenants registrados.
- 4. Atualizar a documentação de onboarding.

9.4 Adicionar uma fixture de estado vazio

- 1. Criar a função de fixture empty para a operação.
- 2. Conectá-la no withProfile da operação correspondente no gateway mock.
- 3. Cobrir nos testes de profile.
- 4. Atualizar a referência de comportamento, trocando "fallback" pelo comportamento real.

9.5 Customizar o tema de um tenant

Editar os campos de theme no descritor do tenant. O provider de tema aplica via CSS vars no próximo render — sem redeploy, pois a configuração é lida em runtime.

Parte 10 — Troubleshooting

10.1 Erros do BFF (HTTP)

HTTP	body.error	Causa	Resolução
400	invalid_sandbox_marker	Começa com app_sandbox_ mas a regex falha	Conferir o formato app_sandbox_<tenant>_<profile>
400	unknown_tenant	Marker referencia tenant não registrado	Usar um tenant válido ou registrar o novo
401	missing_subject	JWT sem claim sub	Pedir ao emissor que inclua o subject
401	invalid_jwt	Token não é marker nem JWT válido	Conferir o que está sendo enviado
403	tenant_mismatch	Em sandbox, marker tenant ≠ header tenant	Alinhar o slug da URL com o tenant do marker
422	Validation failed	Schema de entrada rejeitou o input	Ver body.message para o detalhe
500	mock_gateway_error	Profile error ativo	Esperado em sandbox error; a UI deve tratar

500	Internal server error	Exceção não tratada	Inspecionar os logs estruturados
-----	-----------------------	---------------------	----------------------------------

10.2 Banner não aparece em sandbox

Verificar, em ordem: o token foi de fato injetado no escopo global; o ambiente é de desenvolvimento (o atalho de query param só vale em dev); o marker começa exatamente com `app_sandbox_`; e o loader de bridge está montado.

10.3 Banner aparece em produção (urgente)

Significa que um marker vazou para produção. Investigar qual marker está no escopo global, de onde veio (URL, armazenamento local, código), limpar o armazenamento do parceiro e voltar a usar o JWT real.

10.4 Falha de build por fontes remotas

Quando o firewall bloqueia o provedor de fontes, usar rede com acesso, trocar para fontes locais, ou liberar o domínio no runner de CI.

10.5 Teste de controller falha por contexto não registrado

Controllers injetam o `ExecutionContext`. Nos testes, registrar uma instância antes de resolver o controller:

```
container.registerInstance("executionContext", {
  mode: "live", tenantId: "default", customerId: "test",
} as any);
```

10.6 Registro de DI desatualizado

Use case ou controller novo não aparece no container porque a geração foi esquecida. Regenerar o registro e commitar o resultado.

10.7 Página de tenant falha no prerender

As páginas de tenant precisam de renderização dinâmica. Garantir que o layout do tenant exporte a diretiva de renderização dinâmica.

Parte 11 — Glossário

Termo	Definição
BFF	Backend-for-Frontend. O servidor que renderiza a experiência por tenant.
Core API	API de backend que provê os services de negócio da plataforma.
EmbedView	Widget nativo que embute o BFF em um WebView.
Tenant	Parceiro white-label. Cada um tem um descritor de configuração declarativo.
Marker	String opaca no formato <code>app_sandbox_<tenant>_<profile></code> . Substitui o JWT em sandbox.
Profile	Comportamento do mock em sandbox: happyPath, empty, error, slow.
ExecutionContext	Objeto resolvido na entrada do request com mode, tenantId, customerId, mockProfile?. Propagado via DI.
Resolver	Função <code>resolveExecutionMode(token)</code> que decide sandbox vs live.
Cross-check	Validação sandbox-only: o tenant do marker deve bater com o tenant do header. Mismatch → 403.
MockGateway	Implementação do <code>ICoreGateway</code> que retorna fixtures determinísticas conforme o mockProfile.

CoreHttpGateway	Implementação real do <code>ICoreGateway</code> que encaminha para a Core API.
dataSource	Campo do descritor de tenant que escolhe o gateway default: <code>"mock"</code> ou <code>"core"</code> .
SDUI	Server-Driven UI. UI descrita em templates JSON, interpretada no client.
TemplateNode	Nó da árvore SDUI: <code>{ type, id, props, body, children }</code> .
Dispatcher	Sistema middleware-based que processa eventos de UI.
FlowEngine	Camada de handlers que reage aos eventos do dispatcher (navegação, refresh).
Bridge	Protocolo JavaScript que liga app nativo ↔ webview ↔ web app.
Envelope	Estrutura JSON padrão das mensagens da bridge.
Port	Interface da camada de aplicação. Contrato que a infra implementa.
Gateway	Implementação de um port de acesso a dados (real ou mock).
Use case	Classe da aplicação que orquestra uma operação de negócio.
Controller	Ponto de entrada HTTP — validar → login → executar → responder.
DI container	Container que registra serviços. Root global + child por request.
Tenant theming	Aplicação de cores/logo/layout via CSS vars no <code>:root</code> .
Pull-to-refresh	Gesto nativo + provider de refresh que busca dado fresco e atualiza o contexto.

Parte 12 — Mapa conceitual de componentes

Catálogo dos componentes por subsistema, descritos pela responsabilidade (não pela localização física).

12.1 Autenticação e segurança

- `ExecutionContext` : Tipos do contexto de execução (mode, tenantId, customerId, mockProfile?)
- `parseSandboxMarker` : Parser estrito do marker de sandbox
- `decodeJwtPayload` : Decoder base64url do payload do JWT (sem verificação de assinatura)
- `assertTenantExists` : Valida o slug contra os tenants registrados
- `resolveExecutionMode` : Resolver principal — ramifica entre marker e JWT
- `resolveRequestContext` : Helper de rota — extrai, resolve e faz o cross-check
- `resolveTenant / headers` : Resolução de tenant, CORS e security headers

12.2 Composição (DI)

- `createRequestContainer` : Cria o child container com escopo da request
- Registro de tokens/container : Auto-gerado a partir das classes — não editar à mão
- Gerador de DI : Deriva o registro das classes existentes

12.3 Aplicação

- Ports : 6 contratos (`ICoreGateway` , `ITokenProvider` , `ICache` , `ILogger` , `IConfigRepo` , `ITenantConfigRepository`)
- Use cases : ~17 orquestradores de operação de negócio

12.4 Infraestrutura

- Seletor de fonte de dados : `getTenantConfig` e precedência de gateway
- `CoreMockGateway` : Gateway de mock que honra o mockProfile
- `CoreHttpGateway` : Gateway real que encaminha para a Core API

- `withProfile / MockGatewayError`: Reconciliação de fixtures por profile
- `Fixtures`: Funções de caminho feliz e estado vazio por operação
- `Cache`: Implementação de cache em memória
- `Repositório de config de tenant`: Resolve descritores de tenant
- `Logger`: Logging estruturado

12.5 Interface

- `Controllers`: ~18 pontos de entrada HTTP
- `Schemas de validação`: Validação declarativa de entrada
- `createRequestContext`: Monta o contexto de log (com env)
- `Handlers HTTP`: ~18 rotas no padrão canônico

12.6 Client (web app)

- **SDUI**: `TemplateRenderer`, `data binding`, `composer`, `dispatcher`, `FlowEngine`, `config de refresh`, `schema de template`, `templates`
- **Bridge**: Cliente da bridge, `schema do protocolo`, `loader dev-only`
- **Hooks/UI**: `useSandboxMode`, `hook de prontidão do token`, `SandboxBanner`, `TenantThemeProvider`, `hooks de dados por tenant`
- **Layouts**: `Layout de segmento de tenant` (monta banner + app bar) e layouts por tenant

12.7 SDK nativo

- `EmbedView`: Widget que carrega a URL do tenant no WebView
- `Bridge nativa`: Injeção de token e do script de bridge
- `Message handler`: Tratamento das mensagens da bridge
- `Playground`: App de referência com fluxos sandbox e live

12.8 Documentação e processo

- `Decisão arquitetural do sandbox`: Racional do modo via token marker
- `Arquitetura geral`: Visão das camadas e contratos
- `Contratos de API`: Especificação das operações HTTP
- `Protocolo da bridge`: Envelope, métodos e eventos
- `Referência de sandbox`: Catálogo profile × operação
- `Guia do parceiro`: Onboarding rápido
- `Guardrails de engenharia`: Workflow e regras de segurança

Parte 13 — Exercícios sugeridos

13.1 Conceitual

1. Por que o resolver decide o modo uma vez por request, e não globalmente? Que problema isso resolve?
2. Qual a diferença entre o cross-check de tenant (sandbox-only) e a validação de existência do tenant?
3. Por que SDUI é uma boa escolha para um produto white-label multi-tenant? Quando não seria?
4. Qual a relação entre a fonte de dados do tenant e o marker sandbox? Por que existem os dois?
5. Por que o decoder de JWT é unverified? Isso não é inseguro?

13.2 Prático

1. Coloque o sistema em sandbox com o profile empty. O que muda na home?
2. Acrescente uma fixture de estado vazio para uma operação que hoje cai em fallback. Quantas camadas você toca?
3. Trace uma chamada GET /api/balance do token até a fixture, listando os componentes envolvidos em ordem.
4. Percorra os 4 profiles em sequência. O que muda em cada um?
5. Altere o tema de um tenant e observe o efeito sem rebuild.

13.3 Debug

1. Que erro o BFF retorna quando o marker tem um profile inválido no final? Qual componente o gera?
2. Que erro acontece com um JWT malformado? Como o decoder reage?
3. Por que a suíte E2E está suspensa? O que falta para reativá-la?

13.4 Arquitetura

1. Se o provedor passar a emitir JWTs com claim env: "sandbox" | "live", o que muda para descontinuar o marker? Quantas camadas?
 2. Por que o ExecutionContext é registrado por string no DI, e não por um Symbol como os demais tokens?
-

Parte 14 — Estrutura do Ecossistema e Modelos de Distribuição

Esta seção descreve as estratégias conceituais e práticas para o empacotamento, distribuição e implantação da solução **Agnostic UI** desenvolvida pela **Yuki Labs**. A arquitetura visa garantir o desacoplamento estrito entre as regras do servidor e os clientes, mantendo um ciclo de feedback veloz e baixo acoplamento para os parceiros.

14.1 Topologia do Repositório (Monorepo)

Para manter a sincronização de contratos, tipagens e schemas de validação entre os diferentes subprodutos do ecossistema, o projeto adota uma estrutura de **Monorepo** orquestrada de forma declarativa (utilizando ferramentas como Turborepo ou Nx). Isso permite o versionamento unificado de pacotes internos e garante que qualquer quebra de contrato na Bridge ou nos schemas SDUI seja detectada imediatamente em tempo de compilação.

14.2 Divisão Conceitual de Pacotes (Escopos Yuki Labs)

O ecossistema é quebrado em módulos especializados publicados sob o escopo `@yukilabs`:

`@yukilabs/agnostic-ui-core`

- **Papel:** Núcleo imutável de dados e contratos.
- **Conteúdo:** Tipagens compartilhadas, schemas Zod estruturais (`EnvelopeSchema` , `MockProfileSchema`), utilitários base da *Bridge* e o motor regex de parsing de tokens/markers (`parseSandboxMarker`).
- **Dependências:** Nenhuma (independente de runtime).

`@yukilabs/agnostic-ui-next`

- **Papel:** Motor de infraestrutura e contratos de servidor para rimes Next.js (App Router).
- **Conteúdo:** Abstrações de controllers, middlewares de roteamento dinâmico por tenant, o resolvidor canônico de contexto de execução (`resolveRequestContext`), e os ganchos do gerador automático de Dependency Injection para escopo global e por request (`createRequestContainer`).

`@yukilabs/agnostic-ui-react`

- **Papel:** Interpretador e renderizador client-side para o ambiente web/WebView.
- **Conteúdo:** O componente core `TemplateRenderer` , o interpretador de data binding com expressões e filtros pipes, o barramento de eventos `Dispatcher` ligado ao `FlowEngine` , wrappers estruturais (`TenantSegmentLayout`), além do hook utilitário `useSandboxMode` e do componente visual `SandboxBanner` .

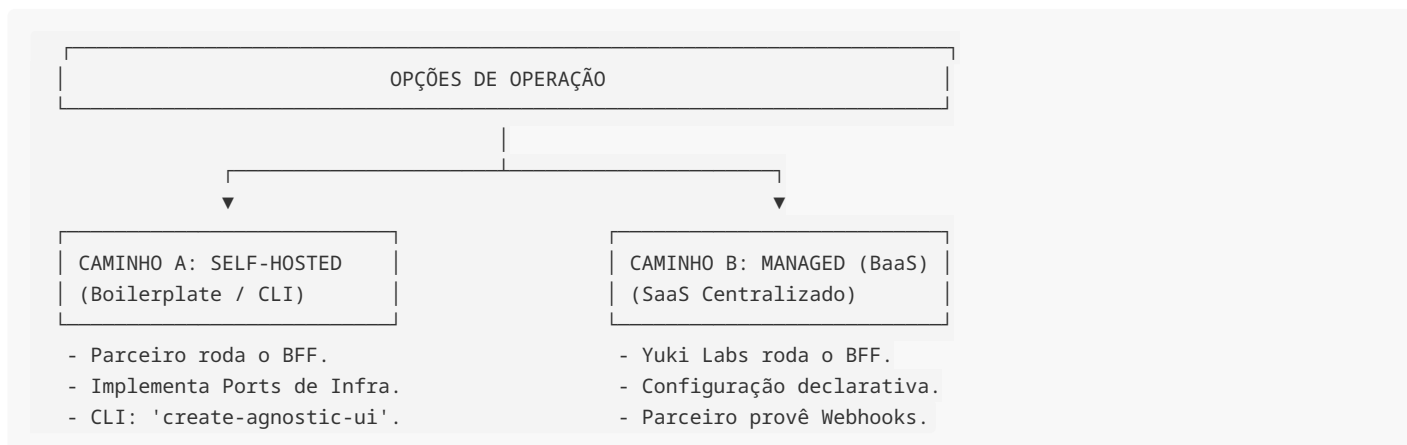
14.3 Distribuição das Pontas Nativas (SDKs)

Para que o parceiro embarque a experiência embarcada com o mínimo de fricção, as camadas nativas são distribuídas através dos gerenciadores canônicos de cada ecossistema:

- **Ecossistema Flutter:** Publicação do pacote `agnostic_ui_sdk` no **pub.dev**. O pacote exporta a `EmbedView` configurada como um widget isolado que recebe o token de acesso e gerencia nativamente o ciclo de vida da bridge e a injeção via canal de plataforma.

- **Ecosistema Nativo iOS:** Disponibilizado como um pacote Swift binário via **Swift Package Manager (SPM)**.
- **Ecosistema Nativo Android:** Publicado como um artefato AAR hospedado no **Maven Central**.

14.4 Modelos de Implantação e Operação



Caminho A — Modelo Self-hosted (Next.js Boilerplate / CLI)

Neste cenário, a Yuki Labs fornece a infraestrutura e os motores de renderização como bibliotecas de prateleira, mas o parceiro é o responsável por subir, hospedar e operar o servidor BFF. * **Mecanismo de adoção:** O engenheiro do parceiro executa um inicializador (`npx create-agnostic-ui`) que monta um projeto Next.js estruturado em Clean Architecture. * **Responsabilidade de Integração:** O parceiro importa as camadas `@yukilabs/agnostic-ui-next` e `@yukilabs/agnostic-ui-core` . Ele fica responsável por codificar a camada de *Infrastructure* concreta, escrevendo a classe que implementa o port `ICoreGateway` para fazer as chamadas HTTP reais contra as suas próprias APIs legadas. * **Casos de uso ideais:** Times de tecnologia maduros que exigem controle absoluto sobre a topologia de rede, latência estrita, segurança corporativa *in-house* e que não podem trafegar dados de clientes por infraestruturas de terceiros.

Caminho B — Modelo Managed / BaaS (Backend as a Service)

Neste cenário, a Yuki Labs opera o BFF Next.js de forma centralizada como uma nuvem multi-tenant SaaS de alta disponibilidade. * **Mecanismo de adoção:** O parceiro não escreve código de servidor. Ele acessa um Portal do Desenvolvedor gerenciado pela Yuki Labs, cria sua conta e preenche o descritor declarativo do tenant (definindo cores, logo, comportamento do app bar e rotas ativas). O portal valida e persiste esse JSON no repositório de configurações de runtime. * **Responsabilidade de Integração:** Para alimentar a interface com dados reais, o parceiro cadastra no portal as URLs dos seus endpoints de negócio (APIs de dados). O motor central da Yuki Labs consome essas URLs atuando como um proxy inteligente, exigindo apenas que os endpoints do parceiro respondam em um formato de contrato JSON padronizado e simplificado estabelecido pela plataforma. * **Casos de uso ideais:** Parceiros que buscam *time-to-market* imediato, custo zero de manutenção de infraestrutura de servidores e atualizações automáticas e transparentes de novos componentes e recursos de SDUI sem necessidade de intervenção de engenharia.

14.5 O Sandbox como Alavanca de Distribuição (Product-Led Growth)

Independente de operar no modelo Self-hosted ou Managed, a engenharia do ecossistema usa o sistema de markers (`app_sandbox_*`) como motor principal de onboarding técnico. Através de um aplicativo utilitário público (Yuki Labs Playground) distribuído nas lojas ou de um emulador web, desenvolvedores conseguem testar e demonstrar layouts inteiros e comportamentos de *profiles* (*empty*, *slow*, *error*) instantaneamente mudando a string do token, validando a arquitetura conceitual antes mesmo de configurar qualquer credencial de segurança ou banco de dados.